

Lab 3

fork, exec & wait

CSCI 4061
Introduction to Operating
System

Sep 14, 2026



UNIVERSITY OF MINNESOTA
Driven to Discover®

fork()

- creates a new process by duplicating the calling process.
 - On success; PID of the child process is returned in the parent process, and 0 is returned in the child process
 - On failure; -1 is returned in the parent, no child process is created, and errno is set to indicate the error

```
pid_t fork(void);
```

```
pid_t getpid(void); // returns current process id
```

```
pid_t getppid(void); // return parent process id
```



fork() - sample/fork.c

```
int main(int argc, char *argv[]) {
    pid_t pid = fork();
    if (pid < 0) { /* Error handling */
        perror("Fork failed");
        return 1;
    }
    else if (pid == 0) { /* Code executed by child */
        printf("Child PID is %d\n", getpid());
    }
    else { /* Code executed by parent */
        printf("This is parent of child PID %d\n", pid);
    }

    return 0;
}
```

```
#include <sys/wait.h>
#include <stdint.h>
#include <stdlib.h>
#include <unistd.h>
#include <stdio.h>
```



wait()

- system call suspends execution of the calling thread until one of its children terminates
 - on success; returns the process ID of the terminated child
 - on failure; -1 is returned
- `pid_t wait(int *stat_loc);`
// takes one argument status and returns a process ID of terminated child.



wait()

```
int main() {
    pid_t pid = fork();
    if (pid < 0) {
        perror("Fork failed");
        return 1;
    }
    else if (pid == 0) {
        printf("Child terminating\n");
        exit(0); /* terminate child */
    }
    else {
        pid_t cpid = wait(NULL); /* reaping parent */
        if (cpid < 0) {
            perror("Wait failed");
            return 1; }
    }
    printf("Parent terminating\n"); }
```

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>
```

Output: Must be in this line order

```
Child terminating
Parent terminating
```



wait() status

```
int status;  
pid_t terminated_pid = wait(&status);
```

WIFEXITED(status): Child exited normally

WEXITSTATUS(status): Return code when child exits

WIFSIGNALED(status): Child exited because a signal was not caught

WTERMSIG(status): Gives the number of the terminating signal

WIFSTOPPED(status): Child is stopped

WSTOPSIG(status): Gives the number of the stop signal



wait() - sample/wait_stat.c

```
1  #include <stdio.h>
2  #include <sys/types.h>
3  #include <sys/wait.h>
4  #include <unistd.h>
5
6  int main() {
7      pid_t pid;
8      int child_status;
9
10     pid = fork();
11     if (pid == -1) {
12         return -1;
13     } else if (pid == 0) {
14         return 1;
15     } else {
16         if (wait(&child_status) == -1) {
17             perror("wait() failed");
18             return 1;
19         }
20         printf("Child exited with status %d\n", WEXITSTATUS(child_status));
21     }
22     return 0;
23 }
```

Output: Assuming regular child termination

Exit status: 1



exec

The exec() family of functions replaces the current process image with a new process image.

```
#include <unistd.h>

int execl(const char *pathname, const char *arg, ...);
int execlp(const char *file, const char *arg, ...);
int execlx(const char *pathname, const char *arg, ...);
int execv(const char *pathname, char *const argv[]);
int execvp(const char *file, char *const argv[]);
int execvpe(const char *file, char *const argv[], char *const envp[]);
```

The exec() functions return only if an error has occurred. The return value is -1, and errno is set to indicate the error.



execvp() - sample/exec.c

```
int main(int argc, char* argv[]) {  
    execvp("pwd", "pwd", NULL); // last argument must be NULL  
                                // Will replace running image with pwd  
    // If execvp returns, it means there was an error  
    perror("execvp failed");  
    return 1;  
}
```

```
execvp("ls", "ls", "-al", NULL);
```



Tasks

- One todo in ***fork_wait.c***:
 1. Wait for child process to complete its execution in parent process
- Two todos in ***fork_exec.c***:
 1. Fork a child process to run the command "*cat sample.txt*"
 2. Parent waits for the child to complete its task

cat is a command line utility that reads data from files, and outputs their contents to the terminal

